PA2 Part 2:
Extending the Shell

SKK UNIVERSITY

PA2

- **Goal**
  - Create shell with pipe & redirection (w7 exercise) support, signals, and job control
  - Adding other built-in commands (bg, fg, jobs, cd, exit, pwd)
    - » bg, fg, jobs are already implemented, but you still need to implement cd, exit, and pwd!
- **Skeleton code will be provided**
  - Since job control is mostly not covered in this class, skeleton code with job control roughly implemented will be provided.
  - You don't have to use it. It just contains functions needed for job control and parsing.
  - The only necessary file to modify pa2.c, but you can modify any part of the skeleton code.

PA2

- **Supplement resources/guides (don't copy the code)**
  - https://sourceware.org/glibc/manual/latest/html_node/Implementing-a-Shell.html
  - https://www.linusakesson.net/programming/tty/
    - » Not a resource on shell but tty (out-of-scope for this course)
    - » Despite being out-of-scope, this page explains job control and signals well.

Example

```
$ curl -s https://www.gnu.org/licenses/gpl-3.0.txt | pa2_head -n 500 | wc -l > output.txt
$ sleep 5 &
$ cat output.txt
0
$ jobs
Job     Group   State   Command
1       391936  running sleep 5 &
$ echo
pa2: Job 1, 'sleep 5 &' has ended

$ sleep a &
$ sleep: invalid time interval `a'
Try 'sleep --help' for more information.
echo $?
pa2: Job 1, 'sleep a &' has ended
1
$ sleep 10
^Zpa2: Job 1 has stopped
$ bg
Send job 1 (sleep 10) to background
$ jobs
Job     Group   State   Command
1       392037  running sleep 10
$ ./bin/pa2
$ exit 10
$ echo $?
10
$ echo
pa2: Job 1, 'sleep 10' has ended

$ echo "Hi" | exit 100
$ echo $?
100
$ exit
```

Shell

SKK UNIVERSITY

Shell

- **Shell**
  - Reads user input, evaluates input, then returns result.
    - » When evaluating input, shell can first lex input, and then parse the result to easily execute the commands later.
    - » Input can contain up to 200 bytes.
    - » Whitespace in input is ignored.
  - For PA2, you will be implementing a shell that is based on bash and fish.
    - » Thus, you can use bash to test the behaviour of your shell but note that not all features in bash are found in your shell (e.g. setting variables, conditionals, loops, etc.)

Shell

- **Grammar (already implemented in parser)**
  - The inputs passed to the shell follow this grammar:

```
input ::= pipeline ["&"];
pipeline ::= commands {"|" commands};
commands ::= command [redirection_in_operator pathname]
                     [redirection_out_operator pathname];
redirection_in_operator ::= "<" ;
redirection_out_operator ::= ">" | ">>" ;
command ::= (executable | builtin) {arg}  ;
arg ::= option | option_with_argument | argument | variable ;
executable ::= prefixed_self_implemented_executable | pathname ;
prefixed_self_implemented_executable ::= "pa2_" self_implemented_executable ;
self_implemented_executable ::= "head" | "tail" | "cat" | "cp" | "mv" | "rm" ;
builtin ::= "cd" | "pwd" | "exit" | "fg" | "bg" | "jobs";
variable ::= "$" variable_name | "~";
```

  - Grammar is in EBNF ([...] = optional, | = or, {...} = ...*)

Parsing structures

- **Input is made up of multiple pipelines**
  - (i.e., `echo “Hi” > file.txt; cat file.txt | wc -l; rm file.txt`)

```
typedef struct {
  Pipeline* first_command_line;
  size_t num_command_lines;
} Input;

typedef struct Pipeline {
  bool is_in_background;
  Command* first_command;
  struct Pipeline* next_command_line;
  uint64_t num_commands;
} Pipeline;
```

Parsing structures

- **Command is a linked list (linked to the next command).**

```
typedef struct Command {
  char* args[MAX_TOKENS]; // argv[0..num_args-1]
  char* stdin; // stdin filename
  char* stdout; // stdout filename
  size_t num_args;
  bool append_stdout; // if redirection is > or >>
  CommandType type; // COMMAND_NULL, COMMAND_BUILTIN,
COMMAND_IMPLEMENTED, COMMAND_OTHER
  struct Command* next_command; // next in pipeline
} Command;
```

Pipeline & Redirection

- **Pipeline**
  - Set of concurrent processes called subprocesses that are connected through standard streams
    - » $ command1 | command2 | command3
      - » command1's output is command2's input
      - » command2's output is command3's input
    - » These processes are independent
      - » `$ echo 1 | cat | exit | echo 2 | cat` returns 2 and does not exit the shell. exit only exits the subprocess.
    - » Do not run these commands "sequentially"; first create a process for each command and wait for all of them to finish
  - Implemented using pipes
  - Associated with a job.

Pipeline & Redirection

- **Pipeline**
  - command1 | command2 | command3
    - » command1's input is STDIN (of shell or redirected STDIN)
    - » command1's output is command2's input
    - » command2's output is command3's input
    - » Command3's output is STDOUT (of shell or redirected STDOUT)
  - All of these commands are running concurrently.
    - » If we have "cat | head –n 2 | cat", what will happen?
      - » First line: a -> goes to first cat
      - » Second line: b -> goes to second cat
      - » Head reads two lines and can safely quit
      - » Second cat is still running since it is expecting input.
      - » After pressing enter, it finds out pipe has been closed and exits.

```
cat | head -n 2 | cat
a
b
a
b
bye
```

Pipeline & Redirection

- **Pipe**
  - Short counts are very normal in pipes as seen in the previous example.
    - » When data is available in STDIN, programs will usually process that and then read STDIN again if applicable.
  - If we have a | b,
    - » What happens if a terminates while b is waiting for input?
      - » `b` will wait for an input and see that the pipe has been closed.
    - » What happens if b terminates and a attempts to write to the pipe?
      - » `a` will receive SIGPIPE and terminate
      - » Example: `cat | exit`
        - » Although cats exits last, the exit code is the exit code of `exit`, not cat

Pipeline & Redirection

- **Redirection**
  - Input redirection (<)
    - » Replaces standard input of process with file in [pathname]
  - Output redirection (>, >>)
    - » Replaces standard output of process with file in [pathname]
    - » If redirected with >, the file is initialized and rewritten
    - » If redirected with >>, append from the end of file (if it exists)
  - If an error occurs due to the file, print "pa2_shell: *pathname*: strerr(*errno*)"

Job Control
(mostly implemented)

SKK UNIVERSITY

Jobs

- **Terminology**
  - Job: “A set of processes, comprising a shell pipeline, and any processes descended from it, that are all in the same process group.” - POSIX Issue 7
  - Job can be a background group or foreground group.
  - Job can have one or more concurrent process(es) via a pipeline (|).
  - A job table is used to keep track of all the jobs, which has a limit of 8192.
  - Note that this means that shell PGID != Job PGID.

Jobs

- **This is how bash holds processes and jobs in the actual code. (jobs.h). This is just to give you an idea on how it is done in actual applications.**

```
typedef struct process {
  struct process *next; /* Next process in the pipeline.  A circular chain. */
  pid_t pid;        /* Process ID. */
  WAIT status;      /* The status of this command as returned by wait. */
  int running;      /* Non-zero if this process is running. */
  char *command;    /* The particular program that is running. */
} PROCESS;

typedef struct job {
  char *wd;    /* The working directory at time of invocation. */
  PROCESS *pipe;   /* The pipeline of processes that make up this job. */
  pid_t pgrp;      /* The process ID of the process group (necessary). */
  JOB_STATE state; /* The state that this job is in. */
  int flags;       /* Flags word: J_NOTIFIED, J_FOREGROUND, or J_JOBCONTROL. */
#if defined (JOB_CONTROL)
  COMMAND *deferred;    /* Commands that will execute when this job is done. */
  sh_vptrfunc_t *j_cleanup; /* Cleanup function to call when job marked JDEAD */
  PTR_T cleanarg;   /* Argument passed to (*j_cleanup)() */
#endif /* JOB_CONTROL */
} JOB;
```

Jobs

- **In skeleton code,**

```
typedef struct Process {
  pid_t pid;
  siginfo_t info;
  bool is_running;
  Command* command;
  int stdin_fd;
  int stdout_fd;
  struct Process* next_process;
} Process;

typedef struct {
  JobState state; // JOB_NULL, JOB_FOREGROUND, JOB_BACKGROUND, JOB_STOPPED,
JOB_DONE
  uint64_t id;
  uint64_t num_processes;
  pid_t pgid;
  Process* first_process;
  Process* last_process;
} Job;

typedef struct {
  Job* table[MAX_JOBS + 1];
  uint64_t num_jobs;
  uint64_t highest_job_id;
} Jobs;
```

Jobs

- **Taken from “The TTY demystified”**

The following shell interactions...

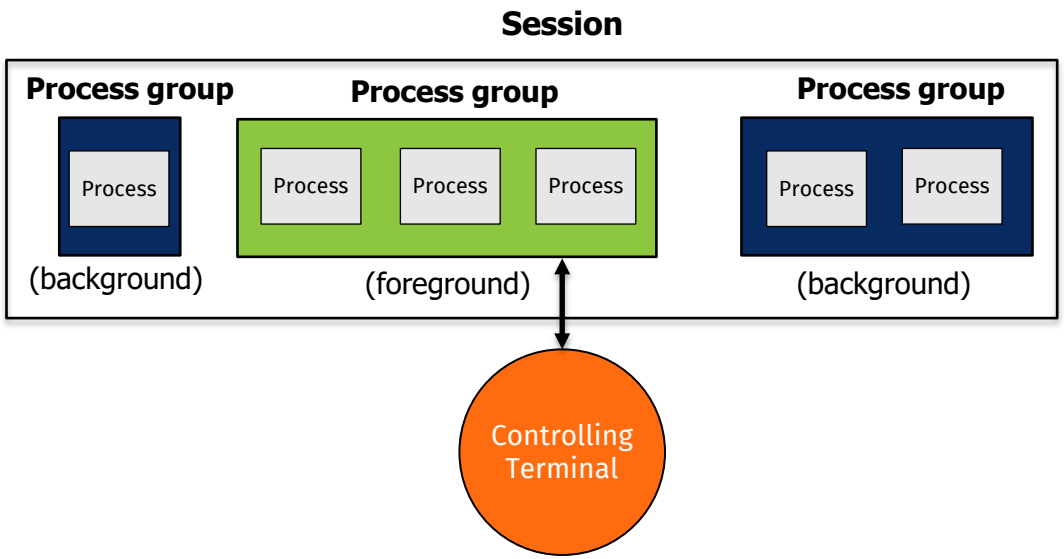
```
Terminal
lft@shizuku:~$ cat
hello
hello
^Z
[1]+  Stopped                 cat
lft@shizuku:~$ ls | sort
```

...correspond to these processes...

Session 100	Session 101			
Job 100	Job 101	Job 102	Job 103	
XTerm (100)	bash (101)	cat (102)	ls (103)	sort (104)
stdin: -	stdin: /dev/pts/0	stdin: /dev/pts/0	stdin: /dev/pts/0	stdin: pipe0
stdout: -	stdout: /dev/pts/0	stdout: /dev/pts/0	stdout: pipe0	stdout: /dev/pts/0
stderr: -	stderr: /dev/pts/0	stderr: /dev/pts/0	stderr: /dev/pts/0	stderr: /dev/pts/0
PPID: ?	PPID: 100	PPID: 101	PPID: 101	PPID: 101
PGID: 100	PGID: 101	PGID: 102	PGID: 103	PGID: 103
SID: 100	SID: 101	SID: 101	SID: 101	SID: 101
TTY: -	TTY: /dev/pts/0	TTY: /dev/pts/0	TTY: /dev/pts/0	TTY: /dev/pts/0

Session in shells

- **Collection of one or more process groups**
  - Session can have a single terminal
    - » This terminal is called a controlling terminal.
  - Process group within a session can be divided into:
    - » A single **foreground** process group
    - » One or more **background** process groups



Foreground processes

- **Synchronous commands**
  - Normal jobs that run in the foreground and control the terminal -> the shell no longer runs in the foreground.
  - `$ grep aladdin Aladdin.txt | sort > sorted.txt`
    - » This single job will run in the foreground.
    - » Each command in the pipeline is referred to as a subprocess.
    - » The first subprocess is the process group leader, but all subcommands are running simultaneously.
  - The exit status is the status of the last command.

Foreground processes

- `int tcsetpgrp(int fd, pid_t pgrp);`
  - Sets the given **pgid** as the foreground process group with the controlling terminal associated with **fd** (i.e. STDIN_FILENO)
  - Has to be called in both parent and child processes.
  - The shell has to regain the controlling terminal after the child has been terminated or stopped.
  - Returns **SIGTTOU** signal to the shell, which will stop it, so it has to be ignored by the shell.
  - A bit tricky to use --- an example code will be added soon

Background processes

- **& (asynchronous commands)**
  - & can be added to the end of a job to make run in the background (i.e. shell does not wait for the job to finish)
  - `$ grep aladdin Aladdin.txt | sort > sorted.txt &`
    - » This single job will run in the background.
  - Returns 0 as an exit status

Signals

- **Signals**
  - Sent to all processes in process group.
  - If a job is currently running, the shell is running in the background, so it cannot receive terminal-associated signals (SIGINT, SIGTSTP). However, it can still receive **SIGCHLD**, which means a child has been stopped or terminated.
    - » This signal can be used to handle (completed/stopped) jobs.
  - Use macros **WIFEXITED**, **WIFSIGNALED along with WTERMSIG**, and **WIFSTOPPED along with WSTOPSIG** to determine if the process has terminated without a signal, terminated due to a signal like SIGINT, or stopped due to a signal like SIGTSTP.

Signals

- **The shell should not terminate/stop when the shell receives these signals:**
  - Ignored signals: SIGQUIT, SIGTSTP, SIGGTIN, SIGTTOU
  - Special behaviour: SIGCHLD, SIGINT
    - » When SIGINT is pending, print a newline.
    - » When SIGCHLD is pending, reap child processes and update job status.
      - » If job is a background job and job has completed, print "pa2: Job JOB_ID, ‘associated command’ has ended“, and then remove the job from the job table and free it.
  - SIGTSTP and SIGCHLD should be set up with SA_RESTART for the sa.sa_flags instead of 0.
- **However, the child processes should still receive these signals (SIG_DFL).**

bg

- **Implemented already but can be used to verify behaviour of SIGCHLD**
- **Synopsis**
  - bg [JOB NUMBER...]
- **Description**
  - "bg sends jobs to the background, resuming them if they are stopped"
  - If JOB NUMBER is not passed, the most recent job would be put in the background.
  - JOB NUMBER format: %1, %2, %3, etc.
  - Print "Send job n to background"
    - » i.e., Send job 1 to background

bg

- **Error**
  - If the job does not exist, print “bg: No suitable job: JOB NUMBER”
  - If the job is not a number, print “bg: JOB NUMBER: invalid integer”
  - If the job table is empty, print “bg: There are no suitable jobs”

fg

- **Implemented already but can be used to verify behaviour of SIGCHLD**
- **Synopsis**
  - fg [JOB NUMBER]
- **Description**
  - fg sends a “job to the foreground, resuming them if it is stopped”
  - If JOB NUMBER is not passed, the most recent job would be put in the foreground.
  - JOB NUMBER format: %1, %2, %3, etc.

fg

- **Error**
  - If there are no jobs that can be put in the foreground, print “pa2_shell: fg: current: no such job”
  - If the given job is invalid, print “pa2_shell: fg: JOB NUMBER: no such job”
  - If the job does not exist, print “fg: No suitable job: JOB NUMBER”
  - If the job is not a number, print “fg: JOB NUMBER: invalid integer”
  - If the job table is empty, print “fg: There are no suitable jobs”

jobs

- **Implemented already but can be used to verify behaviour of SIGCHLD**
- **Synopsis**
  - jobs
- **Description**
  - “jobs prints a list of the currently running jobs and their status”
  - Header: Job\tGroup\State\First command
  - Example:
    - » Job\tGroup\State\First command
    - » 1\t1000\running\sleep 100 &
    - » 2\t1001\stopped\sleep 101

pwd

- **Synopsis**
  - pwd [OPTION]
- **Description**
  - Outputs the current working directory
  - -L prints logical directory (getenv(“PWD”))
  - -P prints physical directory (getcwd())
  - Prints the logical directory by default

pwd

- `char *getcwd(char buf[.size], size_t size);`
  - Gets working directory
  - Return value:
    - » Pointer to string on success. (should be buf)
    - » NULL with errno on failure
  - Size can be set to PATH_MAX (limits.h), which is usually set to 4096 bytes.

cd

- **Synopsis**
  - cd [DIRECTORY]
- **Description**
  - “cd changes the current working directory”
  - If DIRECTORY is not passed or arg[1] is “~", change the directory to the home directory (i.e., getenv(“HOME”))
  - You do not have to set
- **Errors**
  - When directory does not exist, “cd: DIRECTORY: No such file or directory”

cd

- `int chdir(const char *path);`
  - Changes working directory to path.
  - Return value: On success, 0. On error, -1 with errno.
  - Possible Errors:
    - » EACCES: not enough search (execute) permissions for one of the ancestors of path.
    - » ENOENT: directory does not exist
    - » ENOTDIR: ancestor in path is not a directory

exit

- **Synopsis**
  - exit [CODE]
- **Description**
  - exits the shell with CODE if given.
  - If code is outside [0,255], exit with 255.
  - Otherwise, exit with the exit status of the last command in the pipeline (usually 0, but not always).

exit

- **Errors (do not exit)**
  - When code is not a number, `exit: CODE: invalid integer`
  - When more than one argument is passed, print `exit: too many arguments`
  - When job table is not empty, print “There are still jobs active.”
  - Return “exit: There are still running jobs” if there are still active background jobs.